ETHGas

EthgasPool

Security Assessment Report

Version: 3.4

August, 2026

Contents

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the ETHGas components in scope. The review focused solely on the security aspects of the Solidity implementation of the contracts, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the ETHGas components contained within the scope of the security review. The next section summarises the assessment and the identified vulnerability. The detailed finding assigns the vulnerability a severity rating, a status, and a recommendation. The severity rating follows the classification in the Appendix: Vulnerability Severity Classification.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the ETHGas components in scope.

Overview

ETHGas is a marketplace to source and trade blockspace commitments, alongside the Base Fee itself.

This standalone report documents the previously assessed components used by the EthgasPool deployments, particularly:

- `EthgasPool.sol` and `TransferFundHelper.sol` implement pool deposit and operational transfer flows for supported ERC20 assets and native ETH/WETH accounting.
- `ACLManager.sol` manages the administrative, treasurer, timelock, and pauser roles used by the pool.
- `TimelockController.sol` schedules and executes delayed privileged operations.

Security Assessment Summary

Scope

This standalone report consolidates relevant components from two previous ETHGas assessments. The pool transfer components were assessed in the ethgas-developer/ethgas-contracts-core-new-for-audit repository at commit 3fbba35. The remediation of the pool finding was assessed at commit 97c8d63.

The `ACLManager.sol` and `TimelockController.sol` components were previously assessed in the ETHGas Contract Review Security Assessment Report v3. The core contracts in that assessment were initially reviewed at commit b4f8943 and retested at commit bd28876. The development team confirmed that these components have not changed since that assessment. Their original findings remain documented in the corresponding report and are not reproduced here.

The scope represented in this standalone report is limited to:

- `interfaces/IACLManager.sol`
- `interfaces/IEthgasPool.sol`
- `EthgasPool.sol`
- `configuration/ACLManager.sol`
- `dependencies/openzeppelin/contracts/governance/TimelockController.sol`
- `libraries/TransferFundHelper.sol`

Following remediation, the development team provided two sets of Ethereum mainnet deployment addresses for the components represented in this report:

1. **Deployment 1**
   - EthgasPool: 0x191567F92c18fcAFF4559Fcc651992F1F7532449
   - ACLManager: 0x20B2E6Eab939a6934D3e95e1094A488a3d3b70f9
   - TimelockController: 0xFD7e7421E7b0aeb8263f07A082E6a6D942513da3
2. **Deployment 2**
   - EthgasPool: 0xC470772751efAdEa3fb44bF9f3C151721df7a452
   - ACLManager: 0x2826779EA69FCc27DC37AbE51Ea5F054aA145f68
   - TimelockController: 0xcE34F1a842D13a0994618DdBDC7e158F7DB103D8

This report does not represent a new assessment of the deployment configuration.

Note: except for the explicitly listed `TimelockController.sol`, third party libraries, dependencies, deployment scripts, mocks, examples, and generated artifacts were excluded from the scope of the assessments.

Approach

The security assessment covered components written in Solidity.

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

To support the Solidity components of the review, the testing team may use the following automated testing tools:

- Aderyn: `https://github.com/Cyfrin/aderyn`
- Slither: `https://github.com/trailofbits/slither`
- Mythril: `https://github.com/ConsenSys/mythril`

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified one low-severity issue during this assessment.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the ETHGas components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each finding by the matrix given in the Appendix: Vulnerability Severity Classification.

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- ***Open:*** the finding has not been addressed by the project team.
- ***Resolved:*** the finding was acknowledged by the project team and updates to the affected components have been made to mitigate the related risk.
- ***Closed:*** the project team has reviewed and acknowledged the finding, but the recommended remediation has not been implemented. The project team has typically provided a documented rationale supporting this decision, including the approach taken and any associated risk considerations.

Summary of Findings

ID	Description	Severity	Status
EGA-03	Native ETH Transfer Mode Persists Across Batched Pool Transfers	Low	**Resolved**

EGA-03	Native ETH Transfer Mode Persists Across Batched Pool Transfers		
Assets	`contracts/libraries/TransferFundHelper.sol` `contracts/EthgasPool.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Batched pool transfers can send native ETH for later ERC20 rows after an earlier native ETH row, causing wrong-asset transfers and incorrect daily-cap accounting for the affected batch.

The affected `EthgasPool` entry points are:

- `serverPayout()`;
- `serverTransferFundSingle()`;
- `serverTransferFund()`; and
- `serverTransferAnyFund()`.

Each entry point delegates value movement to `TransferFundHelper.serverTransferFund()`.

Inside the helper, `isNativeEth` is declared once before the loop and is set to `true` for native ETH rows, but it is not reset to `false` for later ERC20 rows:

contracts/libraries/TransferFundHelper.sol::serverTransferFund()

```
bool isNativeEth;
IERC20 token;

for (uint i = 0; i < tokenTransfers.length; i++) {
    IEthgasPool.TokenTransfer memory tt = tokenTransfers[i];

    if (tt.token == address(0xEeeeeEeeeEeEeeEeEeEeeEEEeeeeEeeeeeeeEEeE)) {
        isNativeEth = true; // @audit - never reset to false for subsequent ERC20 rows
        tt.token = weth;
    } else {
        token = IERC20(tt.token);
    }

    // ... snip ...
    transfer(isNativeEth, clientAddress, tt, token); // @audit - stale after a prior native ETH row
}
```

Once a native ETH row sets `isNativeEth` to `true`, later ERC20 rows do not reset it to `false`. The transfer helper therefore treats those later ERC20 rows as native ETH transfers:

contracts/libraries/TransferFundHelper.sol::transfer()

```
function transfer(
    bool isNativeEth,
    address clientAddress,
    IEthgasPool.TokenTransfer memory tokenTransfer,
    IERC20 token
) internal {
    if (isNativeEth) {
        (bool success, ) = clientAddress.call{value: tokenTransfer.amount}("");
        if (success == false) revert FailedToSendEth();
    } else {
        token.safeTransfer(clientAddress, tokenTransfer.amount);
    }
    emit IEthgasPool.Withdrawal(clientAddress, tokenTransfer);
}
```

This can surface when a privileged operator submits a transfer list where an earlier row uses the native ETH sentinel and a later row in the same `tokenTransfers` array uses an ERC20 token address. The later row is checked against the ERC20 token's daily cap, but native ETH is transferred if the pool has sufficient native ETH balance.

For example:

1. A treasurer submits a batch containing a small native ETH transfer followed by a larger GWEI transfer.
2. The first row sets `isNativeEth` to `true`.
3. The second row is accounted against the GWEI cap.
4. The second row sends native ETH instead of GWEI because the native ETH transfer mode persists from the first row.

This issue has a low impact as the outcome is an accidental wrong-asset payout that requires operational reconciliation. This issue has a low likelihood as it requires a privileged operator to construct the specific batch ordering where a native ETH row precedes an ERC20 row.

Recommendations

Make the native ETH transfer mode local to each loop iteration, or explicitly reset `isNativeEth` to `false` in the ERC20 branch.

Resolution

The development team updated `TransferFundHelper.serverTransferFund()` to reset `isNativeEth` to `false` when processing ERC20 transfer rows. The fix can be found in commit 97c8d63.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

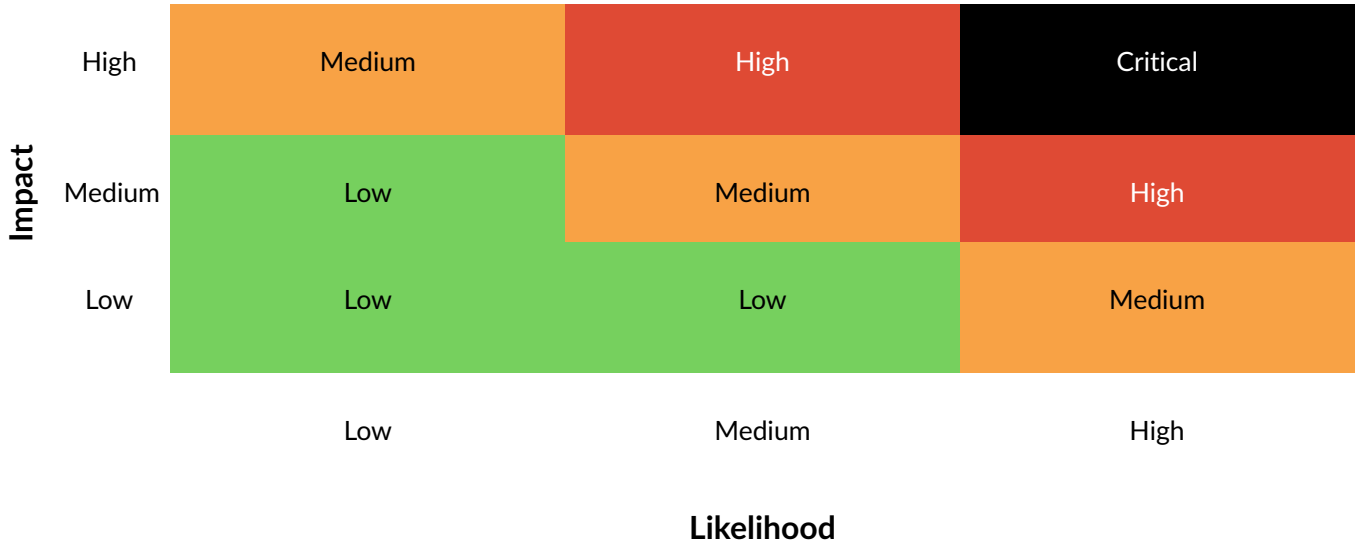

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.